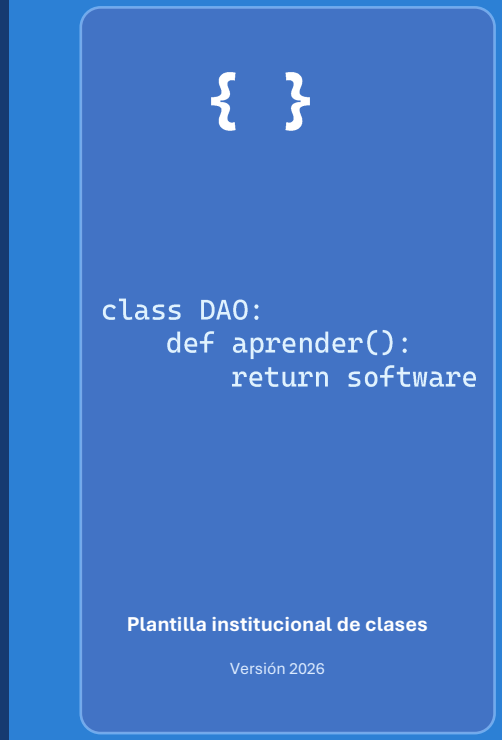
DAO

Desarrollo de Aplicaciones con Objetos

Python • POO • Persistencia • Interfaces • Patrones

UTN – Facultad Regional Córdoba
Ingeniería en Sistemas de Información
Ciclo Lectivo 2026

UTN FRC
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL CÓRDOBA



DAO

Plantel Docente

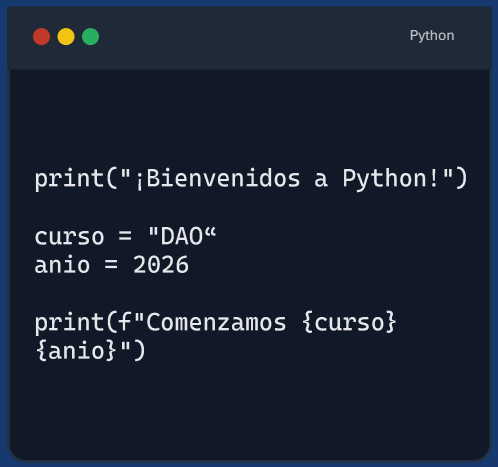


UTN FRC
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL CÓRDOBA

DAO

Temas Clase

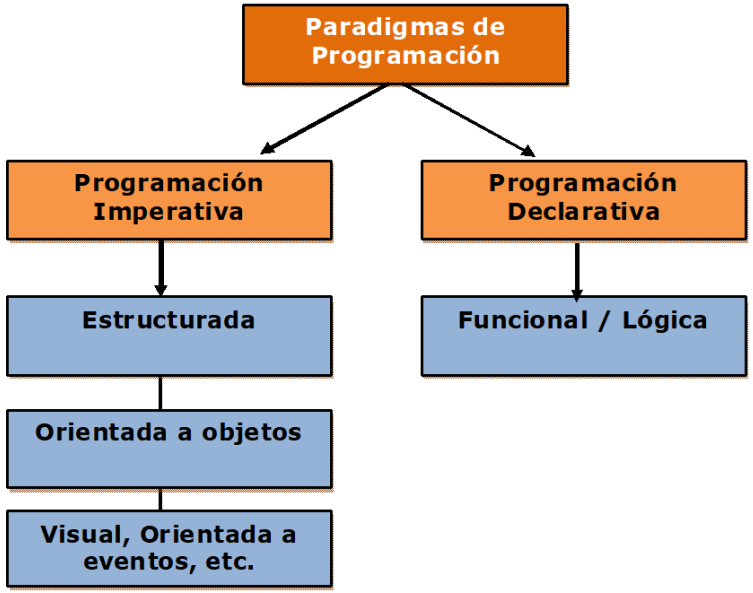
- Clases y Objetos
- Paradigma
- Ejemplos
- Metodos Especiales



1. Paradigmas de programación

Paradigma de programación:

- Modelo básico de diseño e implementación de programas.
- Permite desarrollar software confrme a ciertos principios o fundamentos.

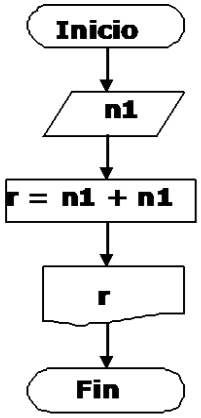


1. Paradigmas de programación

Paradigma Estructurado

Algoritmo + Estructura de Datos = programa





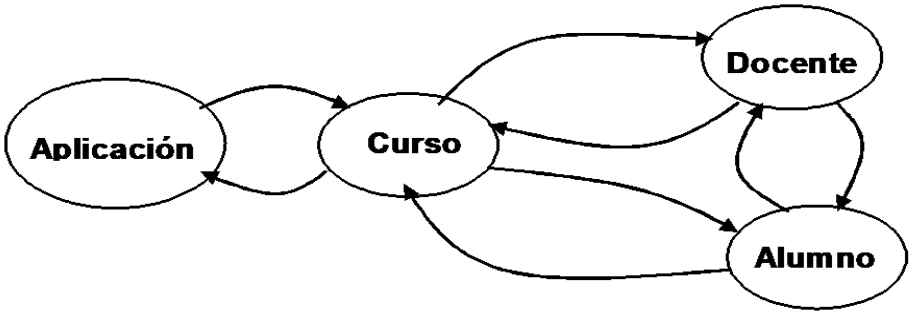

```
Inicio
 Leer n1
 Procesar r = n1 + n1
 Mostrar r
Fin
```

Lenguajes asociados: Fortran, C, Pascal, Python, C++,…

1. Paradigmas de programación

Paradigma Orientado a Objetos

Objetos + Mensajes = Programa



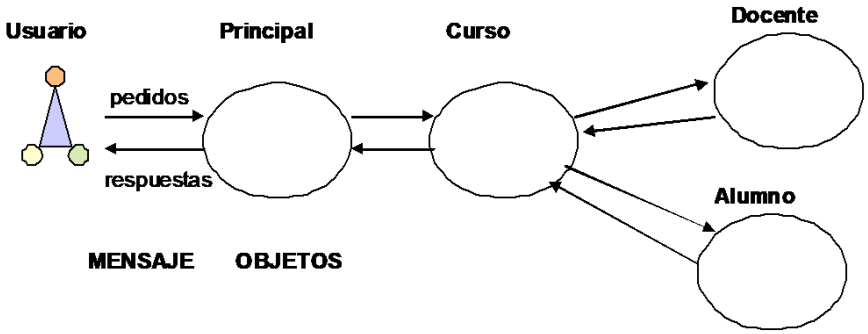
Conjuntos de objetos que colaboran entre si

Lenguajes asociados: Smalltalk, Python, Java, C++,…

2. Paradigma Orientado a Objetos

- Utiliza **objetos** como bloque esencial de construcción.
- Los programas se organizan como colecciones de **objetos** que colaboran entre sí enviándose mensajes.
- Cada **objeto** tiene un rol específico en el programa y todos pueden comunicarse entre sí de formas predefinidas

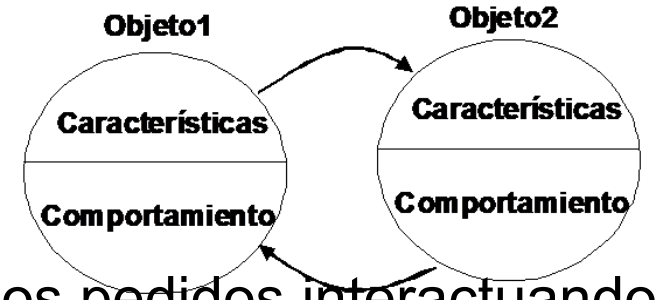




2. Paradigma Orientado a Objetos

Objeto

- ➔ Es una abstracción o representación computacional de una entidad (unidad de software).
- ➔ Posee características (atributos) y comportamientos asociados (métodos).

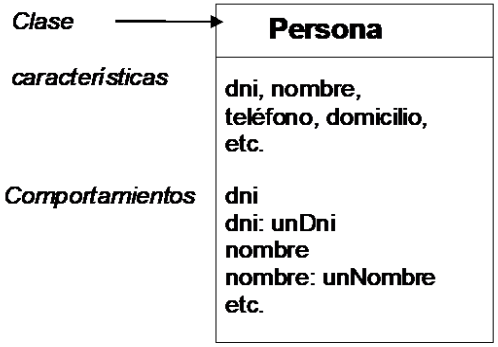


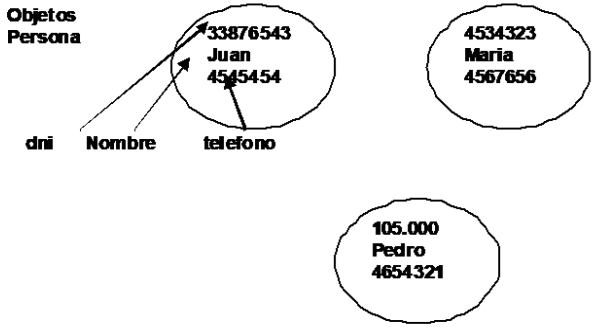
- ➔ Los objetos responden a los pedidos interactuando con otros objetos que conoce, y así se va armando una aplicación.

2. Paradigma Orientado a Objetos

Clase

- ➔ Es un molde, modelo o plantilla para construir objetos.
- ➔ Define las características y comportamientos que serán comunes a todos los objetos de ese tipo.





"Un objeto es una instancia de una clase"

3. Clases en Python

- Las clases proveen una forma de empaquetar datos (atributos o propiedades) y funcionalidad (métodos) juntos.
- Por defecto, los miembros (atributos y métodos), son de acceso público.
- Al crear una nueva clase, se crea un nuevo *tipo* de objeto, permitiendo crear nuevas *instancias* de ese tipo.
- Cada instancia de clase puede tener atributos y métodos para mantener y modificar su estado.
- Sintaxis:

```
class Nombre_Clase:
    #definición de atributos y métodos
```

3. Clases en Python

Clase

El nombre de la clase representa una entidad.
Los atributos son las variables.
Los métodos son las funciones.
Métodos

- **Especiales**: métodos definidos por Python que permiten personalizar determinados comportamientos de los objetos, como `__init__()` y `__str__()`.
- **Básicos**: Modificación (set) y retorno (get) de cada atributo.
- **Negocio o control**: Realizan alguna operación pertinente al comportamiento del objeto.

Clase	**Alumno**
Atributos	legajo, nombre, nota1, nota2
Métodos	__init__ *(inicializador o constructor)* __str__ *(string con todos los atributos)* promedio *(retorna el promedio)* Otros: *set / get para cada atributo* *negocio o control*

3. Clases en Python

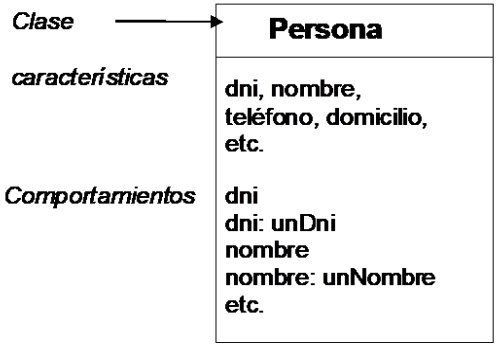
Inicialización: El constructor

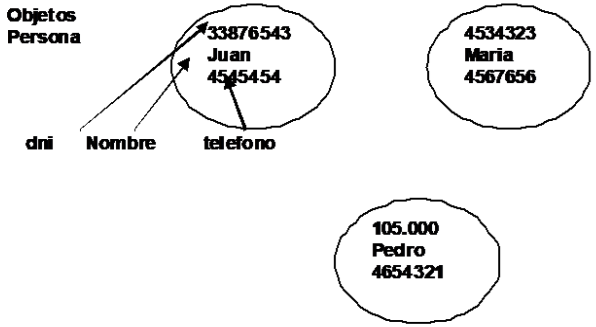
- Es un método que se llama automáticamente
- Tiene la responsabilidad de establecer el estado inicial del objeto
- Es un denominado método “mágico”, es decir se define como __init__:
- Su primer parámetro es una referencia al mismo objeto que está siendo creado.
- Tiene la forma
  - def __init__(self, parametro1, parametro2):

3. Clases en Python

Clase

- ➔ Es un molde, modelo o plantilla para construir objetos.
- ➔ Define las características y comportamientos que serán comunes a todos los objetos de ese tipo.





"Un objeto es una instancia de una clase"

3. Clases en Python

Clase	Alumno
Atributos	legajo, nombre, nota1, nota2
Métodos	__init__ *(inicializador o constructor)* __str__ *(string con todos los atributos)* promedio *(retorna el promedio)* Otros: *set / get para cada atributo* *negocio o control*

```
class Alumno:

    #metodo de inicialización de atributos
    def __init__(self, legajo, nombre, nota1, nota2):
            #atributos
        self.legajo = legajo
        self.nombre = nombre
        self.nota1 = nota1
        self.nota2 = nota2

    #metodo que retorna la cadena con los valores de los atributos
    def __str__(self):
        return "\nLegajo:"+str(self.legajo)+ " -
Nombre:"+str(self.nombre)+\
                " - Nota1:"+str(self.nota1)+ " -
Nota2:"+str(self.nota2)

    #metodo para retornar el promedio del alumno
    def promedio(self):
        promedio = (self.nota1 + self.nota2) / 2
        return promedio
```

4. Objetos en Python

- Son las instancias de una clase.

- Creación:
  - `variable = Nombre_Clase(valor_atr1, valor_atr2, ...)`

- Destrucción:
  - El proceso Garbage Collector, elimina el objeto cuando pierde toda referencia al mismo.

- Invocación de mensajes:
  - `variable.atributo`
  - `variable.metodo()`

4. Objetos en Python

```
def main():

    #crea el objeto y le asigna los valores, invoca al método init
    alu1 = Alumno(12, 'Juan', 7, 9)

    #muestra todos los atributos de alumno, invoca al método str
    print('El alumno es: ', alu1)

    #muestra el promedio, invoca al método promedio
    print('El promedio del alumno es: ', alu1.promedio())

    #cambia el valor del legajo
    alu1.legajo = 15

    #muestra el valor del legajo
    print('El nuevo legajo es: ', alu1.legajo)

    #muestra todos los atributos de alumno, invoca al método str
    print('El alumno es: ', alu1)


main()
```

5. Ejemplos

Clase	Alumno
Atributos	legajo, nombre, nota1, nota2
Métodos	__init__ *(inicializador o constructor)* __str__ *(string con todos los atributos)* promedio *(retorna el promedio)* Otros: *Set/Get para cada atributo* *negocio o control*

1. Usando la clase Alumno del ejemplo anterior, cargar los datos de n alumnos, almacenarlos en una lista, mostrar los datos cargados y calcular el promedio general.

```
#generar la lista vacía  y cargar n alumnos
def cargar_lista(n):
    # genera la lista vacía
    lista = []
    print('Cargue los alumnos:')
    for i in range(n):
        #carga los datos del alumno
        leg = int(input('Legajo: '))
        nom = input('Nombre: ')
        not1 = int(input('Nota 1: '))
        not2 = int(input('Nota 2: '))
        #crea el objeto o instancia alumno
        alu = Alumno(leg, nom, not1, not2)
        #agrega el objeto alumno a la lista
        lista.append(alu)
    return lista
```

5. Ejemplos

```
#generar un string con los
datos de los alumnos
def str_lista(lista):

    listado = '\nLos datos de
los alumnos:'
    listado += '\n '+ ('-' *
50)

    #utiliza for con acceso
directo al objeto
    for alu in lista:
        #muestra los datos de
cada alumno
        listado += str(alu)

    return listado
```

```
#generar un string con los
datos de los alumnos
def get_promedio(lista):
    promedio = sumatoria = 0
    n = len(lista)
    #utiliza for con acceso por
indice a la posicion del objeto
    for i in range(n):
        #accede al metodo
promedio de cada alumno
        sumatoria +=
lista[i].promedio()
    #calcula el promedio
    if n > 0:
        promedio = sumatoria / n

    return promedio
```

5. Ejemplos

```
def main():
    #pide la cantidad de alumnos a cargar
    n = int(input('\nIngrese la cantidad de alumnos a
cargar: '))

    #genera y carga la lista de alumnos
    lista = cargar_lista(n)

    #invoca a las funciones y muestra los resultados
    print('Los elementos de la lista son: ',
str_lista(lista))
    print('El promedio general de los alumnos es : ',
get_promedio(lista))

main()
```

5. Ejemplos

2. Crear una clase llamada Persona, con los atributos: nombre, edad y DNI y los siguientes métodos:

- Un constructor, donde los datos pueden estar vacíos.
- __str__(self): retorna los datos de la persona.
- es_mayor_de_edad(): retorna un valor lógico indicando si es mayor de edad.

```
class Persona:
    #utiliza argumentos por defecto
    def
__init__(self,nombre="",edad=0,dni=""):

        self.nombre=nombre
        self.edad=edad
        self.dni=dni

    def __str__(self):

        return "Nombre:"+self.nombre+" -
Edad:"+str(self.edad)+" - DNI:"+self.dni

    def es_mayor_de_edad(self):
        return self.edad >= 18
```

```
#funcion principal
def main():
    p1 = Persona('Juan', 20,
'43345261')
    print(p1)
    if p1.es_mayor_de_edad():
        print('la persona es mayor de
edad')
    else:
        print('la persona es menor de
edad')

#invocacion
main()
```

5. Ejemplos

3. Usando la clase Persona del punto anterior, crear una clase llamada Cuenta que tendrá los siguientes atributos: titular (que es una persona) y monto (puede tener decimales).
Con los siguientes métodos:

- Un constructor, donde el titular es obligatorio y el monto puede ser 0.
- __str__(self): retorna los datos de la cuenta.
- saldo(): retorna el valor del monto.
- depositar(monto): se ingresa un monto a la cuenta, si el monto es negativo, no se hará nada.
- extraer(monto): se retira un monto de la cuenta, retornar un valor de error en caso de no poder retirar la totalidad.

Aclaraciones: sólo se podrá agregar un titular que sea mayor de edad, tener en cuenta que titular no es un nombre sino un objeto de tipo Persona

5. Ejemplos

```
class Persona:
          ....

class Cuenta:

   def __init__(self,titular, monto =
0):

       self.titular = titular
       self.monto = monto

   def __str__(self):

      return
"Cuenta\n"+"Titular:"+str(self.titular)
+" - Cantidad:"+str(self.monto)

   def saldo(self):
       return self.monto

   def depositar(self,monto):
       operacion = False
       if monto > 0:
           self.monto = self.monto + monto
           operacion = True
       return operacion

   def extraer(self,monto):
       operacion = False
       #invoca al metodo saldo de la clase
par comparar con el parametro monto
       if monto > 0 and self.saldo() >=
monto:
           self.monto = self.monto - monto
           operacion = True
       return operacion
```

5. Ejemplos

```
#funcion principal
def main():
    cuenta = None
    #ingresar los datos para crear la cuenta
    print('Ingrese los datos para crear la cuenta: ')
    nombre = input('Ingrese el nombre del titular: ')
    edad = int(input('Ingrese la edad del titular: '))
    dni = input('Ingrese el DNI del titular: ')

    #crea la persona que va a ser el titular de la cuenta
    titular = Persona(nombre, edad, dni)
    #valida que sea mayor de edad
    if titular.es_mayor_de_edad():
        monto = float(input('Ingrese el monto inicial de la cuenta: '))
        #se crea la cuenta
        cuenta = Cuenta(titular, monto)
        print('Se creo la cuenta con un saldo inicial de: $', cuenta.saldo())
    else:
        print('No se puede crear la cuenta ya que la persona es menor de edad')
```

5. Ejemplos

Resolucion

```
 #verificar que la cuenta exista
   if cuenta is not None:
       #depositar
       monto = float(input('Ingrese el monto a depositar: '))
       if cuenta.depositar(monto):
           print('El saldo actual de la cuenta es: $', cuenta.saldo())
       #extraer
       monto = float(input('Ingrese el monto a extraer: '))
       if cuenta.extraer(monto):
           print('Se pudo extraer $', monto, ' y el saldo actual de la
cuenta queda en: $', cuenta.saldo())
       else:
           print('Fondos insuficientes, NO se pudo extraer $', monto, '
tiene un saldo actual en la cuenta de: ', cuenta.saldo())
   else:
       print('La cuenta no se pudo crear')

#invocacion
main()
```

6. Encapsulamiento

- Es la propiedad de proteger los datos de las clases de manera de controlar su acceso y prohibir modificaciones no permitidas.
- Python no posee modificadores de acceso estrictos como private o protected. Por convención, un nombre que comienza con _ indica que es de uso interno de la clase o módulo. Un nombre que comienza con __ activa un mecanismo llamado name mangling, que dificulta el acceso directo pero no lo vuelve estrictamente privado.
- Entonces, si son privados, ¿cómo puedo utilizarlos?
- Utilizando “setters”, “getters”

7. Métodos especiales

- Son métodos con nombres especiales que se utilizan de manera automática por Python.

Método,	¿Cuándo interviene?	Ejemplo
`__init__()`	Al inicializar un objeto	`Alumno(...)`
`__str__()`	Al representar el objeto como texto	`print(alumno)`
`__eq__()`	Al comparar igualdad	`a1 == a2`
`__gt__()`	Al comparar si es mayor	`a1 > a2`
`__lt__()`	Al comparar si es menor	`a1 < a2`

7. Métodos especiales

- Es la propiedad de proteger los datos de las clases de manera de controlar su acceso y prohibir modificaciones no permitidas.
- Python no posee modificadores de acceso estrictos como private o protected. Por convención, un nombre que comienza con _ indica que es de uso interno de la clase o módulo. Un nombre que comienza con __ activa un mecanismo llamado name mangling, que dificulta el acceso directo pero no lo vuelve estrictamente privado.
- Entonces, si son privados, ¿cómo puedo utilizarlos?
- Utilizando “setters”, “getters”

8. Ejercicios

- **Producto de una tienda**
  Crear una clase Producto que permita almacenar código, descripción, precio y stock. Implementar el constructor y __str__(). Agregar un método hay_stock() que indique si existen unidades disponibles y un método valor_stock() que calcule el valor total de las unidades almacenadas. Crear dos productos y mostrar sus datos.

- **Libro de una biblioteca**
  Crear una clase Libro con título, autor, cantidad de páginas y estado de préstamo. Implementar el constructor y __str__(). Agregar los métodos prestar() y devolver(). No debe ser posible prestar un libro que ya se encuentra prestado. Crear un libro y realizar algunas operaciones para comprobar su funcionamiento.

- **Empleado de una empresa**
  Crear una clase Empleado con legajo, nombre, sueldo y antigüedad en años. Implementar el constructor y __str__(). Agregar un método aumentar_sueldo(porcentaje) y otro método es_antiguo() que retorne True cuando el empleado tenga 10 años o más de antigüedad. Crear al menos dos empleados y probar los métodos.

- **Habitación de un hotel**
  Crear una clase Habitacion con número, cantidad máxima de personas, precio por noche y estado de ocupación. Implementar el constructor y __str__(). Agregar métodos que permitan ocupar y liberar la habitación. Una habitación ocupada no puede volver a ocuparse hasta que sea liberada.

- **Mascota de una veterinaria**
  Crear una clase Mascota con nombre, especie, edad y peso. Implementar el constructor y __str__(). Agregar un método es_adulta() que determine si la mascota tiene 2 años o más y un método actualizar_peso(nuevo_peso) que solamente permita asignar valores mayores que cero. Crear varias mascotas y almacenarlas en una lista.

- **Cuenta de puntos de un cliente**
  Crear una clase Cliente con número, nombre y cantidad de puntos acumulados. Implementar el constructor y __str__(). Agregar métodos sumar_puntos(cantidad) y canjear_puntos(cantidad). No se deben aceptar cantidades negativas ni permitir un canje superior a los puntos disponibles. Crear dos clientes y realizar distintas operaciones.

Clases y Objetos

DAO 2026 · Semana 4